SRI KRISHNA ARTS & SCIENCE COLLEGE KUNIAMUTHUR

FINITE AUTOMATA THEORY IN VENDING MACHINE

BY

DEVISRI.S

17MMA040

II M.SC.MATHEMATICS WITH BIG DATA ANALYSIS

ABSTRACT

Automata theory is dominating in many applications developed from the concept of finite state machine (FSM). This project will consider the design of vending machine (VM), which improves the books delivery service and also design of hypothetical coin-operated vending machine. An efficiency of VM is considered as a problem. In order to improve the efficiency, automata theory depends on the design of the FSM and visual automata simulator (VAS) are used as a method. Finally, the paper will construct FSM that uses fewer states. As a result, we will find that we can improve the efficiency and design cost of the VM.

CHAPTER -1
AUTOMATA THEORY

1.1. INTRODUCTION

Automata Theory is an exciting, theoretical branch of computer science. It established its roots during the 20th Century, as mathematicians began developing - both theoretically and literally - machines which imitated certain features of man, completing calculations more quickly and reliably. The word **automaton** itself, closely related to the word "automation", denotes automatic processes carrying out the production of specific processes. Simply stated, automata theory deals with the logic of computation with respect to simple machines, referred to as **automata**. Through automata, computer scientists are able to understand how machines compute functions and solve problems and more importantly, what it means for a function to be defined as *computable* or for a question to be described as *decidable.*

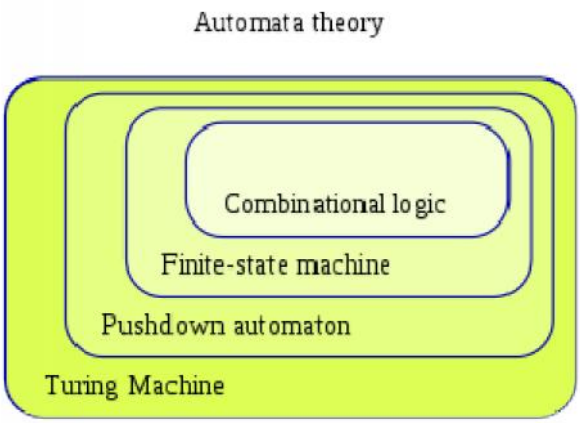
Automatons are abstract models of machines that perform computations on an input by moving through a series of states or configurations. At each state of the computation, a transition function determines the next configuration on the basis of a finite portion of the present configuration. As a result, once the computation reaches an accepting configuration, it accepts that input. The most general and powerful automata is the **Turing machine**.

The **major objective** of automata theory is to develop methods by which computer scientists can describe and analyze the dynamic behavior of discrete systems, in which signals are sampled periodically. The behavior of these discrete systems is determined by the way that the system is constructed from storage and combinational elements. Characteristics of such machines include:

- **Inputs:** assumed to be sequences of symbols selected from a finite set *I* of input signals. Namely, set *I* is the set $\{x_1, x,_2, x_3 ... x_k\}$ where *k* is the number of inputs.
- **Outputs:** sequences of symbols selected from a finite set Z. Namely, set *Z* is the set $\{y_1, y_2, y_3 ... y_m\}$ where *m* is the number of outputs.
- **States:** finite set *Q*, whose definition depends on the type of automaton.

There are **four major families of automaton** :

- Finite-state machine
- Pushdown automata
- Linear-bounded automata
- Turing machine



The families of automata above can be interpreted in a hierarchal form, where the finite-state machine is the simplest automata and the Turing machine is the most complex. The focus of this project is on the finite-state machine and the Turing machine. A Turing machine is a finite-state machine yet the inverse is not true.

1.2. FINITE STATE MACHINE

Automaton

Definition Of Finite State Automata

The exciting history of how finite automata became a branch of computer science illustrates its wide range of applications. The first people to consider the concept of a finite-state machine included a team of biologists, psychologists, mathematicians, engineers and some of the first computer scientists. They all shared a common interest: to model the human thought process, whether in the brain or in a computer. Warren McCulloch and Walter Pitts, two neurophysiologists, were the first to present a description of finite automata in 1943. Their paper, entitled, "A Logical Calculus Immanent in Nervous Activity", made significant contributions to the study of neural network theory, theory of automata, the theory of computation and cybernetics.

Later, two computer scientists, G.H. Mealy and E.F. Moore, generalized the theory to much more powerful machines in separate papers, published in 1955-56.

The finite-state machines, the Mealy machine and the Moore machine, are named in recognition of their work. While the Mealy machine determines its outputs through the current state and the input, the Moore machine's output is based upon the current state alone.

A deterministic finite automaton is represented formally by a 5-tuple $<Q, \Sigma, \delta, q_0, F>$, where:

- Q is a finite set of *states*.
- Σ is a finite set of symbols, called the alphabet of the automaton.
- δ is the transition function, that is, $\delta: Q \times \Sigma \rightarrow Q$.
- q_0 is the *start state*, that is, the state of the automaton before any input has been processed, where $q_0 \in Q$.
- F is a set of states of Q (i.e. $F \subseteq Q$) called accept states.

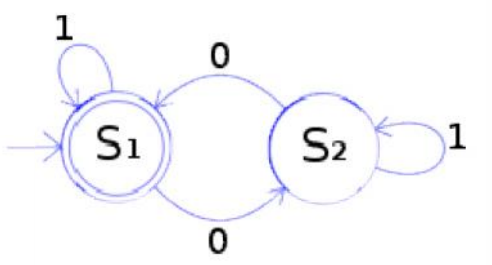
In order to fully understand conceptually a finite-state machine, consider an analogy to an elevator:

An elevator is a mechanism that does not remember all previous requests for service but the current floor, the direction of motion (up or down) and the collection of not-yet satisfied requests for services. Therefore, at any given moment in time, an elevator in operated would be defined by the following mathematical terms:

- **States:** finite set of states to reflect the past history of the customers' requests.
- **Inputs:** finite set of input, depending on the number of floors the elevator is able to access. We can use the set I, whose size is the number of floors in the building.
- **Outputs:** finite set of output, depending on the need for the elevator to go up or down, according to customers' needs.

From the mathematical interpretation above, it can be said that a finite-state machine contains a finite number of states. Each state accepts a finite number of inputs, and each state has rules that describe the action of the machine for ever input, represented in the state transition mapping function.

At the same time, an input may cause the machine to change states. For every input symbol, there is exactly one transition out of each state. In addition, any 5-tuple set that is accepted by nondeterministic finite automata is also accepted by deterministic finite automata.



The study of the mathematical properties of such automata is automata theory. The picture is a visualization of an automaton that recognizes strings containing an even number of 0s. The automaton starts in state S1, and transitions to the non-accepting state S2 upon reading the symbol 0. Reading another 0 causes the automaton to transition back to the accepting state S1. In both states the symbol; 1 is ignored by making a transition to the current state.

There exist **several types of finite-state machines**, which can be divided into three main categories:

- **acceptors**: either accept the input or do not
- **recognizers**: either recognize the input or do not
- **transducers**: generate output from given input

Applications of finite-state machines are found in a variety of subjects. They can operate on languages with a finite number of words (standard case), an infinite number of words (Rabin automata, Bïrche automata), various types of trees, and in hardware circuits, where the input, the state and the output are bit vectors of a fixed size.

1.3. DIFFERENECE BETWEEN DFA AND NDFA

Automata are abstract machines that have a finite set of states. Given some input, they transition from state to state.

- ✓ An NFA is a Nondeterministic Finite Automaton. Nondeterministic means it can transition to, and be in, multiple states at once (i.e. for some given input).
- ✓ A DFA is a Deterministic Finite Automaton. Deterministic means that it can only be in, and transition to, one state at a time (i.e. for some given input).

The major important difference is that an NFA is usually much more efficient.

Each input to a DFA or NFA affects the state of the automaton: if it was in state qq immediately before the input, either it will be in some state q′q′ after the input, or the input will cause it to choke. (that q′q′ may be the same as qq.) Suppose that we have an automaton in a state qq. The difference in behavior between a DFA and an NFA is this:

- If it's a DFA, each possible input determines the resulting state q′q′ uniquely. Every input causes a state change, and the new state is completely determined by the input. Moreover, the automaton can change state **only** after reading an input.
- If it's an NFA, some inputs may allow a choice of resulting states, and some may cause the automaton to choke, because there is **no** new state corresponding to that input. Moreover, the automaton may be constructed so that it can change state to some new state q′q′ without reading any input at all.

As a consequence of this difference in behavior, DFA's and NFA's differ in another very important respect.

- If you start a DFA in its initial state and input some word ww, the state qq in which the DFA ends up is completely determined by ww: inputting ww to the DFA will always cause it to end up in state qq. Hence it is called *deterministic*.
- If you start an NFA in its initial state and input some word ww, there may be several possible states in which it can end up, since some of the inputs along the way may have allowed a choice of state changes. Consequently, you can't predict from ww alone in exactly which state the automaton will finish; hence it is called *nondeterministic*. (And it's actually a little worse, since an NFA is also allowed to have more than one initial state.)

Finally, these differences affect how we determine what words are accepted (or recognized) by an automaton.

- If it’s a DFA, we know that each word completely determines the final state of the automaton, and we say that the word is accepted if that state is an acceptor state.
- If it’s an NFA, there might be several possible final states that could result from reading a given word; as long as at least one of them is an acceptor state, we say that the automaton accepts the word.

Here the view of an NFA that makes it look most like DFA. There is, however, another way of looking at NFAs: it’s also possible to think of an NFA as being in multiple states at once, as if it were making all possible choices at each input. If you think of it in those terms, you can say that it accepts a word provided that at least one of the states in which it ends up after reading that word is an acceptor state. This point of view is perhaps most useful for understanding the algorithm used to turn an NFA into an equivalent DFA.

CHAPTER-2
FINITE STATE AUTOMATA IN VENDING MACHINE

Vending machines (VM) are electronic devices used to provide different products such as snakes, coffee, and tickets, etc. They are designed to be able to accept money and serve product according to the amount of money was inserted. The efficiency of VM depends on the design that uses the automata theory.

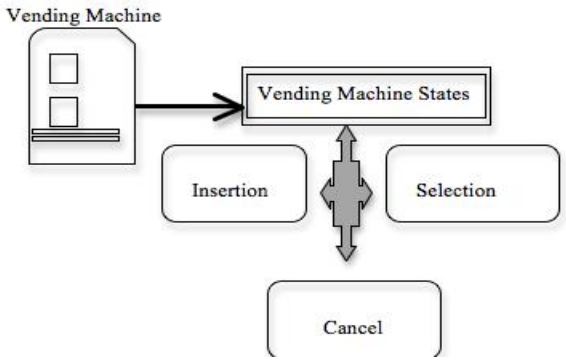
The finite-state automaton is set of all possible states and controls transitions from state to state in response to external input. It also called FSM or finite state automaton (FSA). It is a mathematical tool used to describe processes involving inputs and output. Also, it is suitable for building some different kinds of software, including systems for verifying the correctness of circuits or protocols and the lexical analysis component of a compiler. It is also assisting the control unit in many physical systems including elevators, automatic traffic signals and VM. It can divide FSM into two types: Moore and Mealy state machines. The Mealy machine is also called Synchronous FSM which is a state machine that uses only input actions, and the output depends on the inputs and present state. Moore Machine also called as Asynchronous FSM is a state machine that uses only entry actions, and the output depends on the present state. The efficiency of VM directly influences with the number of states used in the design and the number of steps used in the tools employed.

In this project, we have focused on the efficient design of book VM improved from the basic state diagram based on non- deterministic finite state machine (NFSM). The proposed VM simply can provide four types of books that are:

- English books with 50 rs. Price, Maths books with 50 rs. Price, Notes books with 30 rs. price and Drafts with 20 rs price. The user has to insert the right amount of special type of book and the machine will count and check if correct amount of money inserted and then serve the book directly, the user also can discard and cancel the operation.

2.1. DESCRIPTIONS OF VM AND BASIC DEFINITIONS

Here, description of VM including intelligent approach and challenges are considered. Basically, VM provides several different types of items when money inserted into it. The VMs are more practical, easy to use and accessible for user than the standard purchasing method, they can be found everywhere for different kind of product such as snacks and cold drink, coffee, tickets and diamonds and platinum jewelers. The efficient implementation of these machines can be in different ways by using microcontroller and FPGA board.



2.2. BASIC OPERATION OF VM

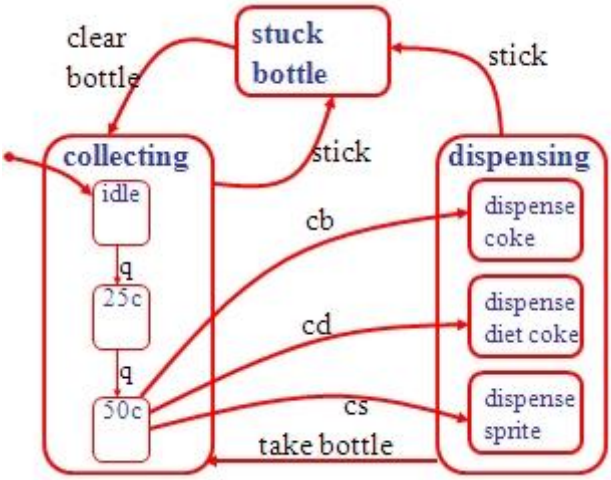
1. The user inserts money, money counter sends to the control unit, the amount of money inserted in the VM by the user.
2. The operation buttons are active to choose the products that people like. According to the VM's internal program, VM dispenses the products when people insert the correct amount.
3. If the program is designed to return the change, VM will return the change.
4. When selected product is not available, VM will reject the service.

VENDING MACHINE IN MARKETS

BART TICKETS VENDING MACHINE

2.3. EFFICIENCY OF INTELLIGENT VM

Energy efficiency is the ultimate target when intelligent VM is in commercial use. Efficiency also confirms that VM can perform their daily functions intelligently with minimum management cost.



2.4. CHALLENGES OF AN INTELLIGENT VM

1- *Opportunities enabled by mobile phones:* Accept payments and transfer operations with the using of communication technology.

2- *Government mandates:* Detail information of medical attentions, weather forecast warnings and policies are displayed through the intelligent VM.

3- *Increased user interactivity:* Interaction through the VM activated from peoples' smart phone which has latest features and facilities.

4- *Remote management:* Exchanging the necessary and secure information during the operation of VM between the vending operators and customers. Through this management, VM operators can decrease maintenance cost and increase machine availability.

5- *Software flexibility:* Compatibility of the features is updated with new technologies and the software.

CHAPTER-3

3.1. ANALYSIS OF VM

1. The user can get four different kinds of books with different prices, named as English book, Maths book, Notes, and Drafts. The prices of these books are given. The machine can accept ten and twenty rupees in any possible sequence, and if the user inserts any type of money different from this, the machine will not accept it. Also, if the user performs any incorrect process such as select unavailable book or inserts money less than the amount, the machine will reject the operation of providing the book.

Number	Price	Book name
1.	50	English book
2.	50	Maths book
3.	30	Note
4.	20	Draft

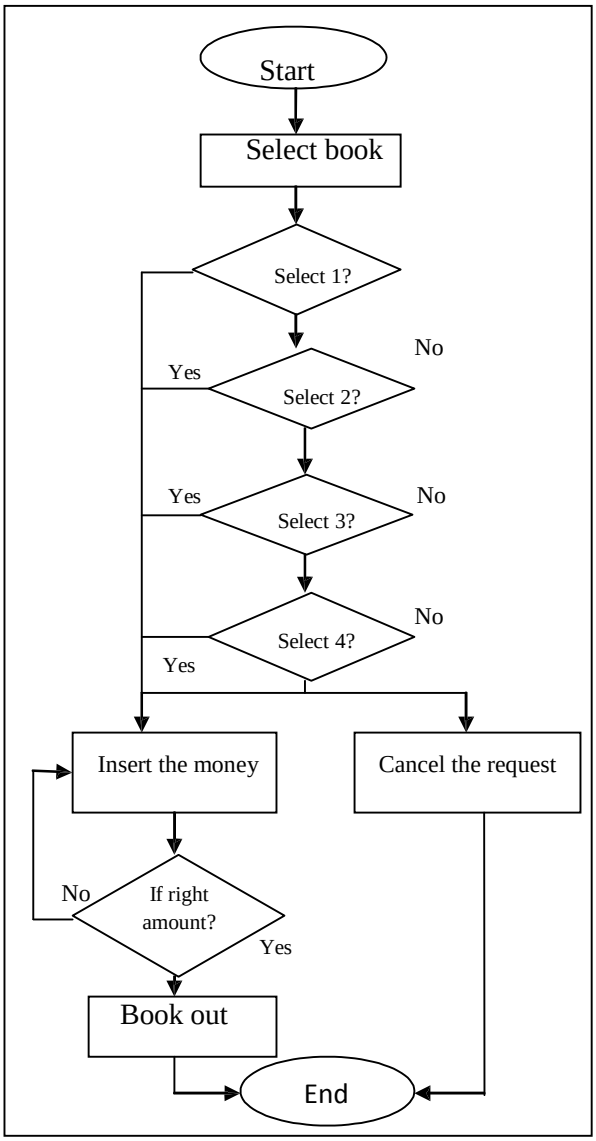
There are four select (select1, select2, select3, select4) inputs are taken for selection of products. Namely SELECT1, SELECT2, SELECT3 and SELECT4. SELECT1 used for the selection of English book. Similarly SELECT2, SELECT3 and SELECT4 represents an Maths book, Note and Draft respectively. The user inserts money of two types 10 and 20 rs, 1 and two inputs represent 10 and 20 rs respectively. A cancel (c) input is used when the user wants to cancel his request. When the user cancels the request or adds more money than the desired money, the proposed VM cannot provide money for return or change. Because the money will not return the user should cancel his request only before inserted any money. The proposed VM is designed using FSM modeling, we decide to use Nondeterministic Finite Automata (NFA), because; it does not require input symbols for state transitions and is capable of transitioning to zero or more states for a given start state and input symbol.

This distinguishes it from a deterministic finite automaton (DFA), in which all transitions are uniquely determined and in which an input symbol is required for all state transitions. So (NFA) enabling the user to select multiple choices from one state easily, the detail of the entire data the proposed machine is shown below.

Input	Description
E	Select 1 for English books Transitions
A	Select 2 for Maths books Transitions
N	Select 3 for Note books Transitions
D	Select 4 for Drafts Transitions
R	Reset = back to initialize state

C	End process = go to cancel the state
1	10 SR
2	20 SR

The state diagram consists of the following states (initial state, user Selection, amount of money the user inserted). The initial state is the first state of the design; the proposed machine will be ready for the users to select the book. After the initial state, the user will select the book. Then the transition will go into one of the four states (select1, select2, select3 and select 4) depended on the book the user chose. After that the machine will be waiting for the user to insert money, and the machine can accept only two types of (10 and 20) rs. When the desired amount is inserted, the machine will go to the accepting state. When the user select English book will go to a state then user insert a sequence of money the machine will go to specific state depending on the total amount of money as example the user enter 10, 20 and 10 rs, the machine will go to "10 rs " state then to "30 rs" state and go to "40 rs" state, At any moment, the machine has to decide whether or not to deliver the selected book or not.



3.1.1. Implementation And Results

Simulations

VAS tool is used to describe design methodology. It is a helpful tool for simulating, showing and transforming FSA. VAS tool was designed with a simple way to use and clear visualization of how FSA (DFA and NFA) work. Represent the behavior of the machine for all possible sequences of money. Any state has tow circles inside each other are accepting state: As soon as the machine reaches this state, the book delivers. Thus, for four books and two types of rupees (10 and 20), there are twenty states. The price of the Maths book and the English book is the same.

Thus, the transition from Select 2 goes to the same transition of select

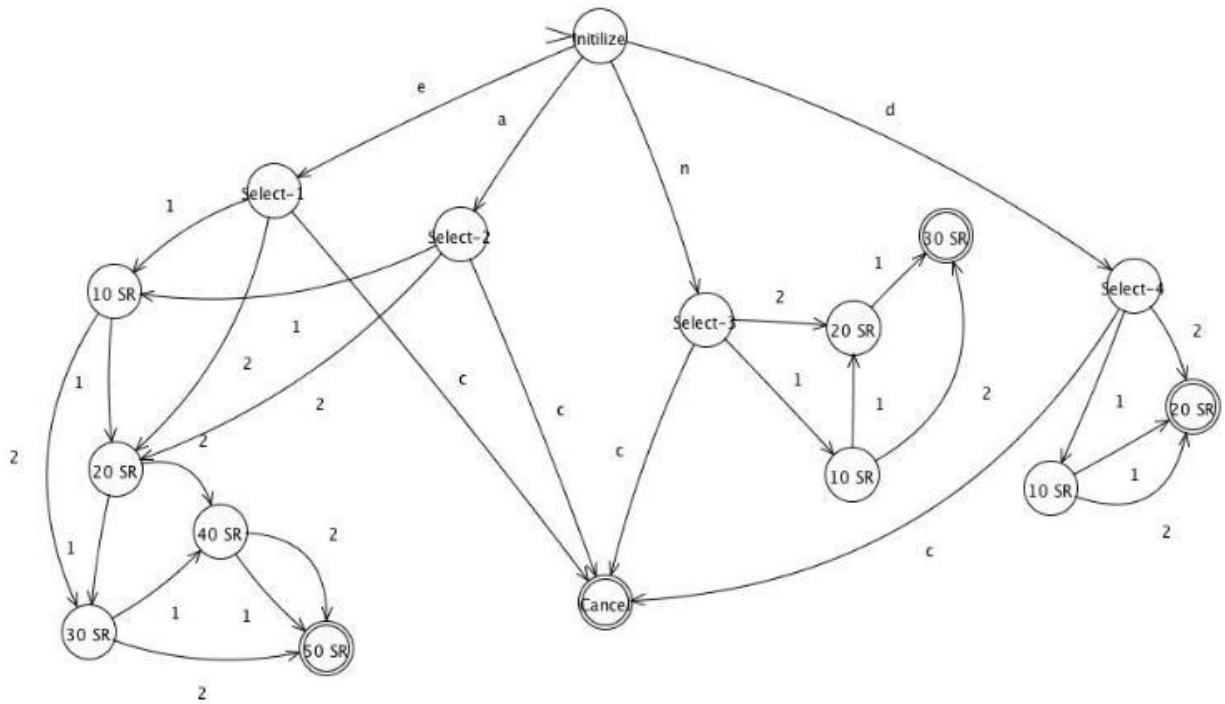
1. So the state diagram only shows the 16 states excluding the five states of select
2. This is an important issue in developing proposed VM with fewer states. The machine will work much faster with using fewer states.

3.1.2. Implementation Of Accepting Operation

The user should select a specific book and enter 10,10,10,10 and 10 for example in order to get the Arabic book. Transition details for the serving book show these transitions for a serving book using the book vending. The string is a11111b. The Final b inserted refer to the end of the operation; it is only referred to the debug steps for reach accept state. After the selection of book type, the user has to insert the money, which is the amount of the specific book, and in our explanation example equal to 50 rs. At the first time, the user may insert 10 rs and continue, or the user can use a single payment.

Transition details for serving Book

Selected string a11111b		
Symbol	**Remaining string**	**States**
A	11111b	[Select 2], accepting = false
1	1111b	[10SR], accepting = false
1	111b	[20SR], accepting = false
1	11b	[30SR], accepting = false
1	1b	[40SR], accepting = false
1	b	[50SR], accepting = True


Initilize
e
a
n
d
Select-1
Select-2
Select-3
Select-4
10 SR
20 SR
40 SR
30 SR
50 SR
Cancel
30 SR
20 SR
10 SR
20 SR
10 SR
c
1
2

3.2. COIN-OPERATED VENDING MACHINE DESIGNED USING CONCEPTS IN AUTOMATA THEORY:

2. Here designed a mathematical model for a hypothetical coin- operated vending machine. A vending machine is an electronic machine used to disperse a product, which is usually a beverage and/or snack items, to a consumer for a certain amount of money the hypothetical vending machine disposes products with the following price values:

1. 15 for the first item
2. 25 for the second item
3. 30 for the third item

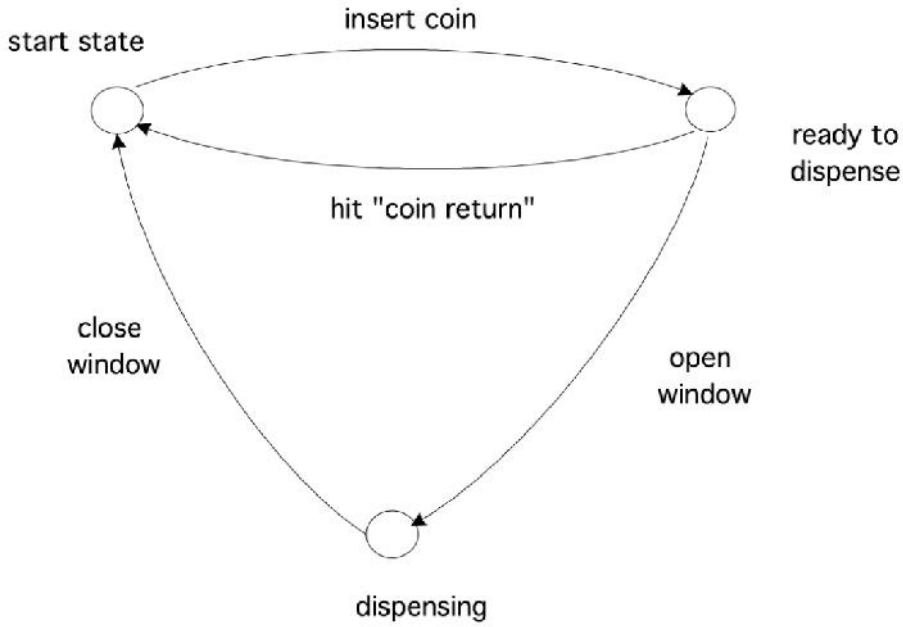
Thus, we have three accepting states. The said values could be reached by inserting a finite number of the following tokens:

1. token value of 5 (denoted by a)
2. token value of 10 (denoted by b)
3. token value of 20 (denoted by c)

In our model, we have the following elements:

1. $Q = \{q_1, q_2, q_3, q_4, q_5, q_6, q_7\}$
2. Start state $q_0 = q_1$
3. $F = \{q_4, q_6, q_7\}$
4. $\Sigma = \{a, b, c\}$
5. $\delta: Q \times \Sigma \rightarrow Q$.

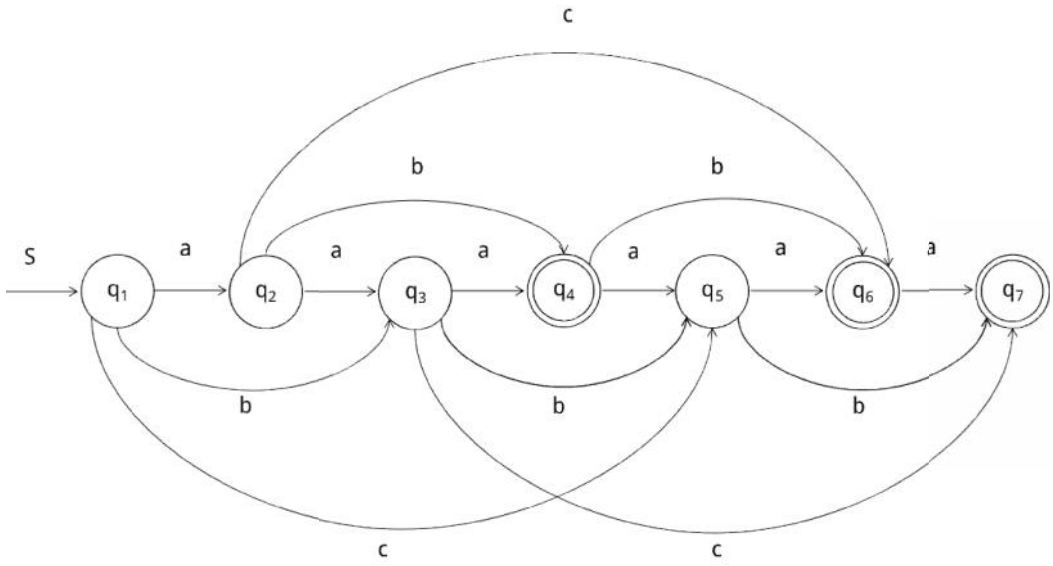
$\delta(q_1, a) \rightarrow q_2$		
$\delta(q_2, a) \rightarrow q_3$	$\delta(q_1, b) \rightarrow q_3$	
$\delta(q_3, a) \rightarrow q_4$	$\delta(q_2, b) \rightarrow q_4$	$\delta(q_1, c) \rightarrow q_5$
$\delta(q_4, a) \rightarrow q_5$	$\delta(q_3, b) \rightarrow q_5$	$\delta(q_2, c) \rightarrow q_6$
$\delta(q_5, a) \rightarrow q_6$	$\delta(q_4, b) \rightarrow q_6$	$\delta(q_3, c) \rightarrow q_7$
$\delta(q_6, a) \rightarrow q_7$	$\delta(q_5, a) \rightarrow q_7$	



From our previosly-defined transition functions (δ), we can draw the following table:

States (Q)	a	b	c
→ q_1	q_2	q_3	q_5
q_2	q_3	q_4	q_6
q_3	q_4	q_5	q_7
*q_4	q_5	q_6	e
q_5	q_6	q_7	e
*q_6	q_7	e	e
*q_7	e	e	e

The arrow symbol (→) denotes the start state q_0, the asterisk symbol (*) denotes the final or accepting states, and e denotes an empty state. The elements we have just laid down thus defines the following transition diagram:



Graphical representation of the deterministic finite automaton (DFA) for the hypothetical machine.

Since we already have a DFA model for our machine, it is no longer necessary to design a non-deterministic finite automata (NFA). This is because an NFA is a finite-state machine as well, with the capability "to be in several states at once". Simply put, an NFA can be exactly identical to a DFA, that is why it is no longer necessary for us to design a separate NFA.

3.2.1. Implication

The studies in automata theory are abstract machines describe, as well as solving the computational problems are much easier and understandable than the other approaches. The logical understanding of input and output process depends on the study of operating virtual machines. Automata theory researches are classified as a recognizable formal language, which help the researcher to improve such application in artificial intelligent, compiler design, and other fields of the computer since that depend on computational theory.The implication of VM depends on the number of features, which must be efficient to both customer and service providers. Increasing efficiency is dependent on the appropriate design of FSM employed in the future VM. Increasing facilities based on customer expectations,
technology, time, etc.

CONCLUSION

The proposed book VM design is studied using NFSM automata theory. Constructing FSM, which uses fewer states that enable the machine to provide fast response serving. The improvement of the efficiency and design cost was considered. Using this tool and appropriate design, the efficiency of VM can easily be enhanced for many applications. Complexity reductions that involve with the time and space increase the efficiency of the overall system used in VM.

The future work of this VM is to improving it by adds more books and scale the type of products it included according to students needs. Also, we suggest that the machine design to be able to accept ATM cards instead of paper money. Also, improving efficiency and complexity reduction by using appropriate automata machine will be possible in order to convert the VM to an intelligent approach.

BIBLIOGRAPHY

REFERECE BOOKS:

1. Ashwag Alrehily, Ruqiah Fallatah and Vijey Thayananthan, Design of Vending Machine using Finite State Machine and Visual Automata Simulator, International Journal (0975-8887)
2. J. Holub and S. Stekr. On parallel implementations of deterministic finite automata. In Implementation and Application of Automata, CIAA ’09, pages 54–64, 2009.
3. Varkey, M. and Sunny, J. (2014). Design and Implementation of Multi Select Smart Vending Machine. International Journal of Computer Networks and Wireless Communications (IJCNWC), ISSN: 2250-3501
4. Vijey Thayananthan, Ashwag Alrehily and Ruqiah Fallatah, “Automata Design with Time Complexity for Intelligent Vending Machine based on Visual Automata Simulator”. IEEE 17th International Conference on Computer Modelling and Simulation, UK 2015, University of Cambridge, pages 159-164

WEBSITES:

- Deterministic finite automaton https://en.wikipedia.org/wiki/Deterministic_finite_automaton.
- Vending Machine. http://www.businessdictionary.com/definition/vending-machine.html.
- https://www.researchgate.net/.../276136463_Design_of_Vending_Machine_using_Finite_Automaton
- https://www.ijcaonline.org
- web.cs.ucdavis.edu/~rogaway/classes/120/spring13/eric-dfa